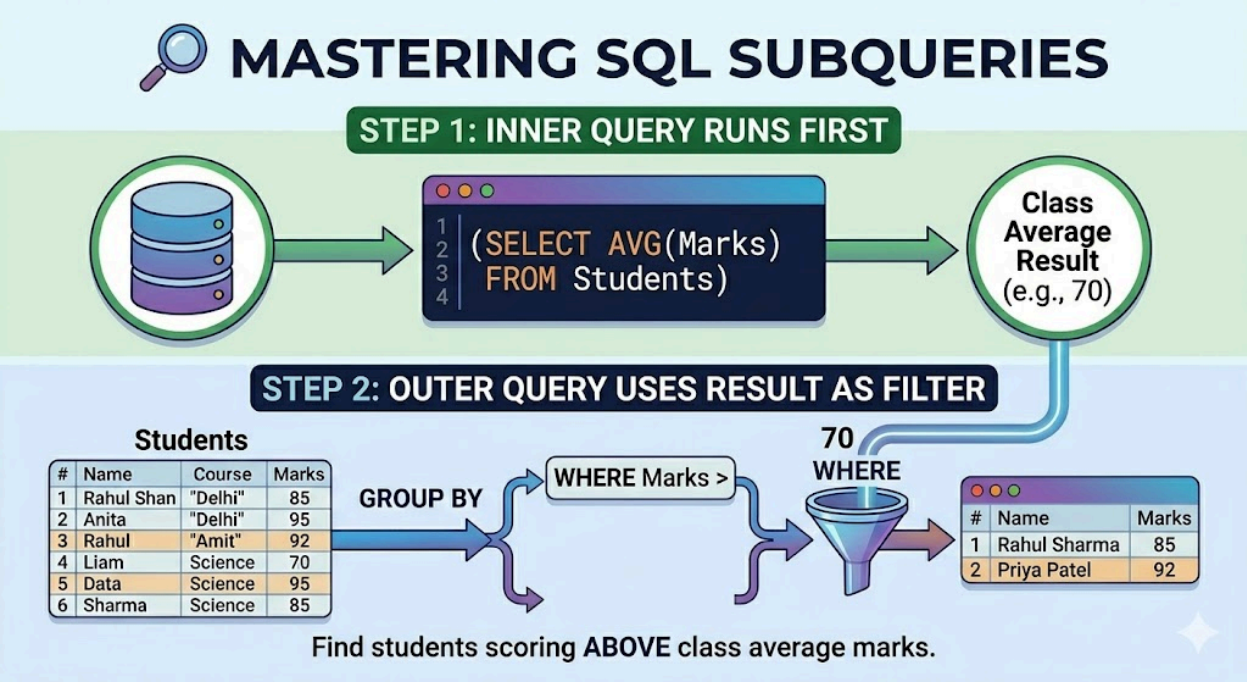
# Day 3: Subqueries, Capstone Project

Welcome to Day 3 of the SQL Bootcamp! Today, we are diving into advanced query techniques that will allow you to perform complex, multi-step calculations and apply conditional logic directly within your database. We will end the session with a mini-project and a final assignment to test everything you've learned.




## 🔍 1. Subqueries

**What subqueries are and when to use them**

A subquery (or nested query) is a query placed inside another SQL query. You use subqueries when you need the results of one query to act as a filter or a calculation input for your main query. Think of it as answering a question before you can answer your main question.

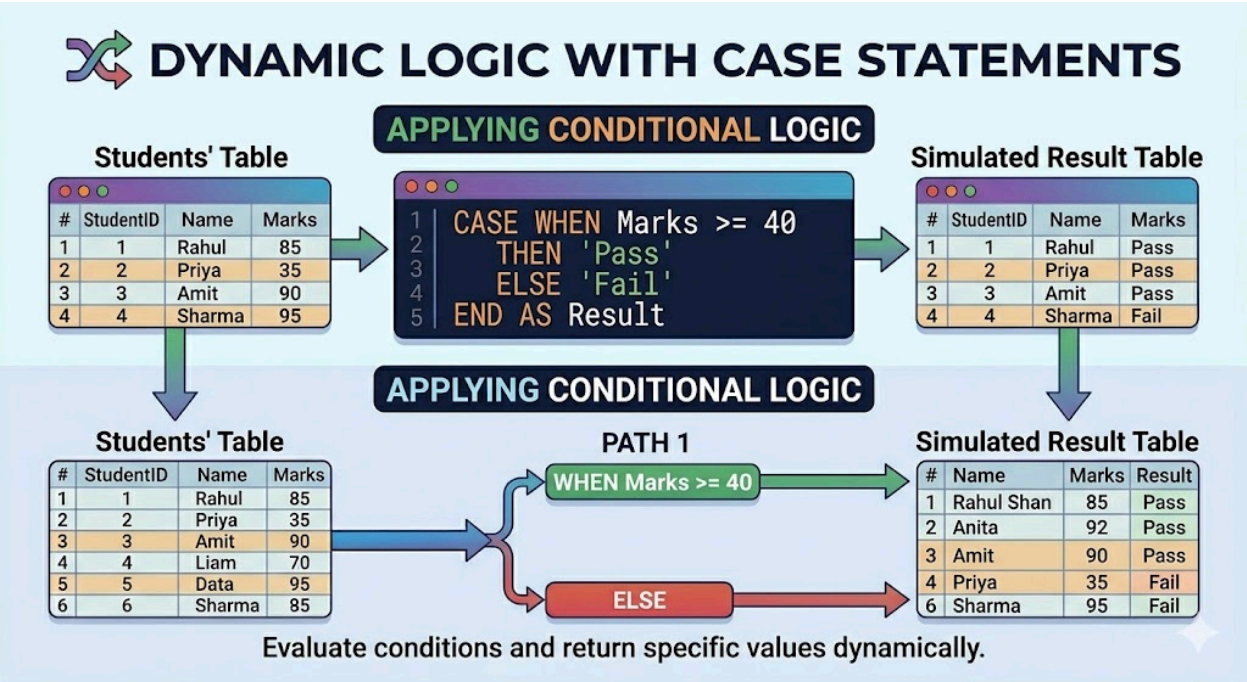
**Example: Find students scoring above average marks**

Imagine you want to find all students who scored above the class average. You

can't just write `Marks > AVG(Marks)` in a `WHERE` clause. Instead, you use a subquery to first calculate the average, and then compare each student's marks to that result.

SQL

- `- The query inside the parentheses runs first to find the averageSELECT Name, Marks FROM Students WHERE Marks > (SELECT AVG(Marks) FROM Students);`




## 🔀 2. CASE Statements

**Conditional logic inside queries**

The `CASE` statement is SQL's way of handling "if/then/else" logic. It allows you to evaluate conditions and return specific values depending on whether those conditions are met.

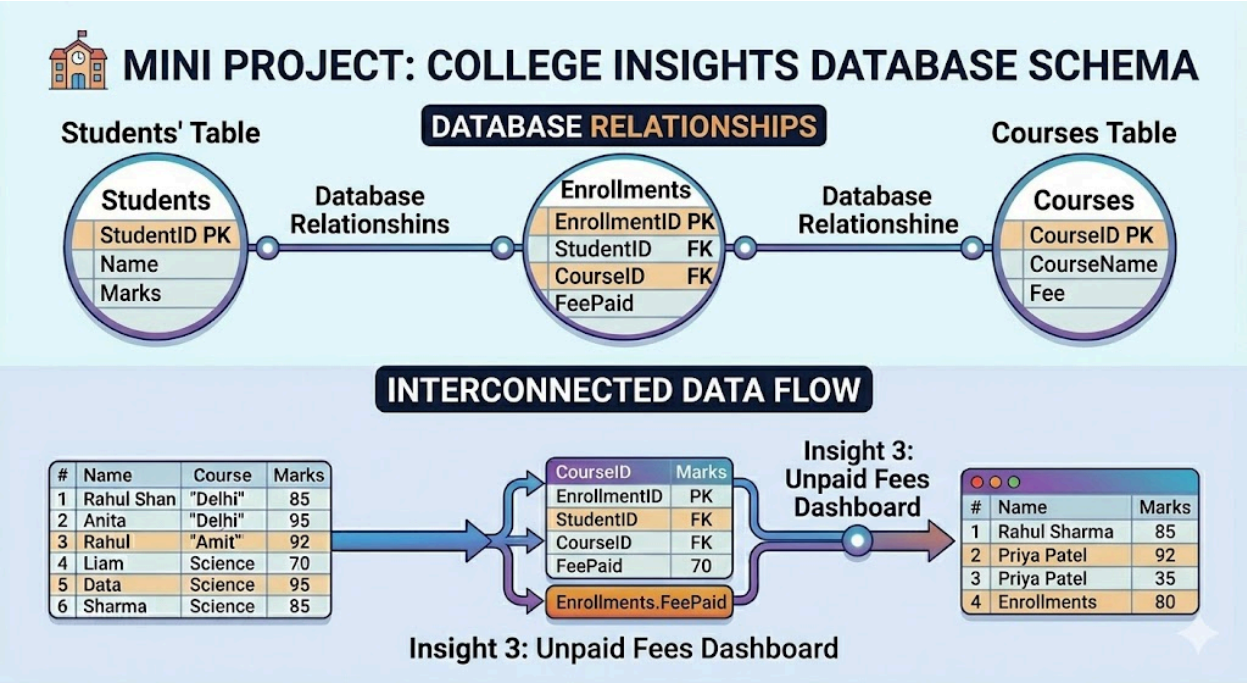
**Example: Show "Pass" or "Fail" based on marks**

If you want to categorize student performance dynamically without permanently altering your table data, `CASE` is the perfect tool.

SQL

```
SELECT
    Name,
    Marks,
    CASE
```

```
        WHEN Marks >= 40 THEN 'Pass'
        ELSE 'Fail'
    END AS Result
FROM Students;
```




## 🏫 3. Mini Project: College Insights Dashboard

Now, let's put these concepts to work. For this mini-project, we will build 3 tables: `Students`, `Courses`, and `Enrollments`.

*Assume you have created these tables and populated them with sample data.*

1. Find total students per course

SQL

```
SELECT Courses.CourseName, COUNT(Enrollments.StudentID) AS Total_Students
FROM Courses
LEFT JOIN Enrollments ON Courses.CourseID = Enrollments.CourseID
GROUP BY Courses.CourseName;
```

2. Calculate average marks per course

SQL

```
SELECT Courses.CourseName, AVG(Students.Marks) AS Average_Marks
FROM Students
INNER JOIN Enrollments ON Students.StudentID = Enrollments.StudentID
INNER JOIN Courses ON Enrollments.CourseID = Courses.CourseID
GROUP BY Courses.CourseName;
```

3. Identify students who haven't paid fees *(Assuming the Enrollments table has a* `FeePaid` *boolean/status column).*

SQL

```
SELECT Students.Name, Courses.CourseName
FROM Students
INNER JOIN Enrollments ON Students.StudentID = Enrollments.StudentID
INNER JOIN Courses ON Enrollments.CourseID = Courses.CourseID
WHERE Enrollments.FeePaid = 'No';
```